An Intrusion Detection Scheme for Internet of Vehicles Based on Non-IID Federated Distillation and DQN-PBFT

Wenxian Jiang, *Senior Member, IEEE*, Zhenping Guo, and Naizhou Wang

***Abstract*—The Internet of Vehicles (IoV) is rapidly evolving, but it also faces significant security threats, including Denial of Service (DoS) attacks and deceptive behaviors. The construction of an intrusion detection framework for the IoV encounters the following challenges: 1) the pervasive Non-IID (Non-Independent and Identically Distributed) issue within datasets and 2) in existing applications combining Federated Learning (FL) with blockchain, node load issues are often overlooked. To address these challenges, we propose a novel scheme that integrates Federated Distillation (FD) with blockchain technology to safeguard the IoV from network attacks. First, the scheme combines Convolutional Neural Networks (CNN) and FD for intrusion detection, which both preserves the privacy of vehicle users' data and effectively shares knowledge. Next, for each FD client, we construct a selector based on density ratio estimation to filter out accurate and reliable knowledge from local predictions, thereby effectively mitigating the performance degradation caused by the Non-IID data problem. Then, the DQN (Deep Q-Network) integrated PBFT algorithm (DQN-PBFT) is proposed to reduce the number of consensus nodes using the DQN algorithm, enhancing load balancing and fairness during the FD and block generation processes. Finally, experimental results show that, in a Non-IID environment, the proposed scheme outperforms the baseline scheme by approximately 30% in accuracy. Compared to other consensus algorithms, DQN-PBFT demonstrates greater advantages in FD environments, exhibiting higher consensus efficiency.**

***Index Terms*—Internet of Vehicles, federated distillation, blockchain, DQN, intrusion detection.**

I. Introduction

With the rapid advancement of computing and communication technologies, the prevalence of connected and autonomous vehicles (CAVs) in the modern world is steadily increasing. Internet of Vehicles (IoV) technology provides a crucial communication framework for autonomous vehicles, ensuring reliable communication with other IoT entities, such as infrastructure, pedestrians, and smart devices. The fifth-generation (5G) and upcoming sixth-generation (6G) networks are expected to revolutionize the IoV by offering ultra-reliable communication with ultra-low latency and high bandwidth.

As the connectivity and complexity of the IoV grow, cybersecurity risks have become a major concern. The emergence of new technologies has provided attackers with more sophisticated tools, enabling them to launch novel attacks such as DoS, data injection, and information theft [1]. Therefore, there is an urgent need for efficient Intrusion Detection Systems (IDS) to protect the IoV from DoS attacks and other intrusion threats. Intrusion defense systems play a pivotal role in ensuring network security and are considered the first line of defense against cybersecurity threats. IDS are designed to detect these security threats effectively and in a timely manner, while maintaining a low false positive rate. However, traditional intrusion defense systems, which rely on pattern/signature detection [2], struggle to identify zero-day attacks (i.e., unforeseen or anomalous attacks). Moreover, substantial effort is required to continuously improve these systems to detect zero-day attacks. Recently, the advent of machine learning and deep learning (ML/DL) technologies has transformed many fields, such as visual and speech recognition, surpassing human performance in some areas. By applying ML/DL techniques, IDS can effectively detect intrusion activities and security threats in network traffic [3]. These ML/DL-based IDS can identify both existing and novel attacks without the need for constant rule updates required by traditional IDS. However, due to the lack of balanced and up-to-date labeled training datasets, these ML/DL-based IDS perform poorly when facing emerging security threats such as zero-day attacks [4]. The privacy nature of datasets and the widespread presence of adversarial attacks hinder vehicle organizations and the research community from sharing their sensitive datasets, preventing the development of efficient and up-to-date ML/DL models to address emerging security threats.

Federated Learning (FL) has emerged as a subfield of ML, aimed at training a shared global model across multiple distributed clients without the need to centrally store sensitive data. In FL, each client trains a local model using local data and then only sends local model updates instead of uploading sensitive data to a central server. This significantly enhances

Received 1 January 2025; revised 1 June 2025; accepted 29 August 2025. This work was supported in part by the Xiamen Science and Technology Project under Grant 3502Z20251018, in part by the Excellent Teaching Case Project of Graduate Degree of Fujian Province under Grant 00489054, and in part by the Fundamental Research Funds for the Central Universities of Huaqiao University's Academic Project under Grant 2024HQYJ01. The Associate Editor for this article was S. H. Islam. *(Corresponding author: Wenxian Jiang.)*

Wenxian Jiang is with the College of Computer Science and Technology, Huaqiao University, Xiamen 361000, China, and also with the School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China (e-mail: jwx@hqu.edu.cn).

Zhenping Guo and Naizhou Wang are with the College of Computer Science and Technology, Huaqiao University, Xiamen 361000, China (e-mail: 1821630622@qq.com; wangnaizhou2000@163.com).

Digital Object Identifier 10.1109/TITS.2026.3682598

the efficiency of traditional ML/DL-based IDS in dealing with network attacks while protecting the privacy of collaborators. However, FL typically relies on a centralized architecture with a single central server, which exposes it to the risk of a single point of failure. The application of blockchain can effectively mitigate the risk of single-point failures in FL, thereby ensuring the security of the global model. Moreover, merely detecting intrusion behaviors is insufficient; the analysis of intrusion logs is crucial for intrusion response, vulnerability analysis, and forensics. The security of intrusion logs should not be overlooked, as they are at risk of tampering. As a result, some studies have integrated FL with blockchain to establish intrusion detection frameworks [5]. However, traditional FL requires frequent uploads of DL model parameters, leading to significant communication overheads that severely hinder its practical deployment in the IoV. Worse still, various attacks can recover clients' original data from the uploaded model parameters, posing an ongoing risk of information leakage [6].

Recently, a Federated Distillation (FD) scheme [7], [8], [9] has been proposed to reduce the communication overhead and security risks associated with model parameter exchanges. In FD, the prediction results (local logits) from clients on a public dataset are aggregated into global logits. By utilizing hard or soft labels (i.e., predicted results) from public samples rather than model parameters, FD significantly reduces communication overhead, enhances privacy protection, and supports heterogeneous local models. However, since FD relies on the aggregation of local predictions for distillation rather than on trained local models, it is particularly sensitive to the training state of local models and is vulnerable to issues arising from insufficient or poor-quality training. Additionally, the Non-IID (Non-Independent and Identically Distributed) data problem between clients exacerbates this challenge, especially when there is a large discrepancy between the distribution of public data and local data, which may cause local models to make inaccurate predictions.

To address the challenges faced by existing IDS in the IoV environment, this paper proposes a novel framework. The framework combines CNN and FD techniques, enabling precise identification of network attacks while effectively protecting vehicle privacy and reducing communication overhead. Additionally, a density ratio estimation-based selector is designed for each FD client to filter out sample predictions that might mislead other clients. To ensure the security and distributed nature of the framework, blockchain technology is introduced, and a DQN-integrated PBFT algorithm is proposed, which not only improves consensus efficiency but also enhances load balancing during the FD and consensus processes among nodes. The main contributions of this paper are summarized as follows:

- To address the issues of network attacks and high communication overhead, we propose an intrusion detection scheme for the IoV that combines CNN with FD. This scheme utilizes an unlabeled public dataset and generates labels for the unlabeled data by exchanging model outputs, enabling each local model to further train and enhance its performance. Moreover, Federated Distillation mitigates the communication overhead by exchanging model outputs instead of directly exchanging model parameters, effectively avoiding the communication burden associated with larger models.
- To tackle the decline in model performance caused by the Non-IID issue in IoV data, we design a density ratio estimation-based selector for each FD client to identify out-of-distribution samples in the public dataset. This method quantifies the density differences between in-distribution and out-of-distribution samples, effectively detecting anomalies. When the density ratio of a sample falls below a predefined threshold, the client treats it as an outlier (i.e., the sample is inconsistent with the client's data distribution) and avoids sharing its prediction, thereby preventing it from misleading other clients.
- To improve the efficiency and decentralization of FD, we integrate blockchain technology into the framework. Specifically, we propose a DQN-integrated PBFT algorithm (DQN-PBFT), which takes into account key parameters such as the communication capacity and computational resources of FD clients to intelligently select suitable clients as consensus nodes. By reducing the number of nodes involved in consensus, we can significantly enhance consensus efficiency. Furthermore, this mechanism also improves load balancing and fairness in the FD and blockchain processes among clients.
- Through experiments, we compare the proposed FD scheme with the baseline in terms of accuracy, precision, and F1-Score. The experimental results show that the FD scheme significantly outperforms the baseline, with an accuracy improvement of approximately 30%. Additionally, we compare DQN-PBFT with existing consensus schemes in terms of throughput, delay, and communication overhead. The experimental results demonstrate that DQN-PBFT performs better in the FD environment and achieves higher consensus efficiency compared to existing schemes.

The structure of this paper is organized as follows: Section II reviews the related work; Section III presents the network model we propose; Section IV provides a detailed explanation of the federated distillation scheme; Section V describes the system framework; Section VI evaluates the performance of the proposed scheme through experiments; and finally, Section VII concludes the paper.

II. Related Work

In recent years, the development of IDS in the IoV environment has gained significant attention, as it is crucial for ensuring the security and privacy of these connected systems. Many researchers have proposed various IDS schemes (as shown in Table I). Yu et al. [10] proposed an IDS for detecting intrusion attacks in the automotive CAN network using a time interval conditional entropy fuzzy method. This approach identifies and detects attacks by collecting and analyzing the conditional entropy values of regular communication messages. Deng et al. [11] introduced a practical IDS based on creating voltage fingerprints for each message ID. The system can detect intrusions without prior knowledge of the secret mapping between ECUs and IDs, and it is capable

TABLE I
EXISTING RESEARCH SOLUTIONS

Reference	Main idea	Classifier	Dataset	Performance evaluation
Yu *et al.* [10]	A time-interval conditional entropy-based intrusion detection system for automotive CAN	Conditional entropy reference value of messages	CAN-intrusion-dataset	Accuracy
Deng *et al.* [11]	A voltage-based IDS	Voltage fingerprint established for each message ID	Custom dataset	Accuracy, F1-score, Precision
Du *et al.* [12]	CLUSTER method for CAN bus intrusion detection	Euclidean distance	Car-Hacking Dataset provided by HCRL	Accuracy, F1-score, Recall, Precision
Korium *et al.* [13]	A machine learning-based IDS for detecting abnormal behavior by inspecting network traffic	RF, XGBoost, CatBoost, LightGBM	CIC-IDS-2017, CSE-CIC-IDS-2018, CIC-DDoS-2019	Accuracy, F1-score, Recall, Precision
Waghmode *et al.* [14]	It combines an exhaustive feature selection algorithm with a LS-SVM classifier to improve detection accuracy and reduce false positives.	LS-SVM	NSL-KDD, CIC-IDS-2017, UNSW-NB15	Accuracy, F1-score, Recall, and Precision
Lo *et al.* [15]	CNN and LSTM used to automatically extract spatial and temporal features from vehicular network traffic	CNN, LSTM	Car-Hacking Dataset provided by HCRL	Accuracy, Precision, Recall, F1-score, FPR, FNR
Agrawal *et al.* [16]	LSTM-CNN combined with reconstruction and threshold techniques to detect various attacks	CNN, LSTM	Car-Hacking Dataset provided by HCRL	Precision, Recall, F1-score
Li *et al.* [17]	Proposes two transfer learning-based intrusion detection model update schemes to address emerging attacks in the IoV	Ensemble learning	AWID dataset	Accuracy, Detection rate, False alarm rate, FNR
Abou *et al.* [18]	FL for model training and privacy protection, decentralized security reputation system based on blockchain to maintain the reliability and credibility of FL in ITS	Pysyft project [19]	UNSW-NB15 dataset	Accuracy, Precision, Recall, F1-score
Boualouache *et al.* [20]	Deep neural network combined with open set recognition, using blockchain technology for trustworthy decentralized FL	DAE, Deep-MCDD	5G-NIDD dataset, VDoS	Accuracy, Precision, TPR, FPR, F1-score, AUROC
Beuran *et al.* [21]	FL combined with shrink autoencoder and centroid classifier is adopted to identify various IoT attacks through reconstruction error	Centroid anomaly detection	N-BaIoT dataset	AUC

of detecting malicious frames with periodic or non-periodic IDs. As a self-learning IDS, it adapts to different in-vehicle networks without the need for customization for different vehicle models. However, these traditional non-ML systems struggle with detecting zero-day attacks, as they often rely on predefined rules or signatures, which are prone to being bypassed or misjudged.

ML models, on the other hand, can continuously improve and adjust themselves by learning new data and attack patterns. This means they can adapt to the ever-changing threat landscape. Du et al. [12] transformed CAN bus intrusion detection into an open set recognition problem and proposed the CLUSTER method. By using the distance from known categories to cluster centroids as the training loss, they ensure consistency with the open set recognition threshold. By learning intra-class compactness and inter-class separability, CLUSTER effectively classifies known attacks and identifies unknown ones. Korium et al. [13] proposed an ML-based IDS that identifies anomalous behavior by analyzing network traffic. The system first uses Z-score normalization for data preprocessing, preserving data distribution and handling outliers. Then, it employs machine learning algorithms such as Random Forest (RF), Extreme Gradient Boosting (XGBoost), CatBoost, and Light Gradient Boosting Machine (LightGBM) for model training. The model achieves over 99.8% accuracy on three merged datasets by optimizing hyperparameters to control the training process and prevent overfitting. However, traditional ML methods typically require expert manual feature engineering, which is time-consuming and may miss critical features. Moreover, these methods may underperform when handling large datasets and are susceptible to overfitting or underfitting. Waghmode et al. [14] proposed a machine learning-based network intrusion detection method that combines an exhaustive feature selection algorithm with a least squares support vector machine (LS-SVM) classifier. The study focuses on addressing key challenges faced by traditional intrusion detection systems in big data environments, such as feature redundancy and high false alarm rates. A supervised learning framework is employed to build an efficient and secure model. This method emphasizes balancing detection accuracy and computational efficiency through feature selection and classifier optimization to meet the demands of real-time threat detection in complex network environments.

DL methods can automatically extract high-level features from raw data, reducing the reliance on manual feature engineering. At the same time, when handling large-scale datasets and complex tasks, DL models typically demonstrate higher accuracy and better generalization capabilities. Lo et al. [15]

proposed a hybrid deep learning intrusion detection system (HyDL-IDS) based on spatiotemporal representations. This system sequentially uses CNN and LSTM networks to automatically extract spatial and temporal features from in-vehicle network traffic. Compared to other approaches, HyDL-IDS significantly improves detection accuracy and reduces false positive rates in detecting intrusions in vehicular networks. Agrawal et al. [16] proposed an intrusion detection system based on deep learning (LSTM, CNN), combining thresholding and error reconstruction methods. The authors trained and explored various neural network architectures and compared their performance. In addition, they provided a reconstruction error distribution graph to visually demonstrate the system's ability to distinguish between normal and abnormal sequences. However, DL approaches also have some drawbacks, particularly due to the lack of balanced and up-to-date labeled training datasets, leading to poor performance when facing zero-day attacks. Li et al. [17] proposed two transfer learning-based intrusion detection model update schemes to address the continuously evolving attack types in the IoV, each applicable to different scenarios. When the IoV cloud can provide a small amount of labeled data, a cloud-assisted update scheme is adopted to improve detection accuracy through transfer learning. When the IoV cloud cannot provide labeled data, vehicles use pseudo-labels to perform multiple rounds of local transfer learning and fuse the models to achieve local updates.

As a result, recent studies have applied FL to IDS in IoV. FL not only leverages the latest dataset knowledge from clients but also effectively protects clients' privacy. Abou et al. [18] proposed an edge-computing-based framework utilizing FL and blockchain technology to protect intelligent transportation systems (ITS) from zero-day attack threats. This framework includes a distributed edge computing architecture and a decentralized reputation system, ensuring system security and privacy through collaboration and reputation management. Experimental results demonstrate that this framework is efficient and accurate in threat detection. Boualouache et al. [20] modeled the expected communication patterns of connected autonomous vehicles (CAVs) using Deep Auto-Encoder (DAE) and detected any deviations from this pattern as malicious activity. They also employed open set recognition and FL to train an attack classifier (AC) model, which is a deep multi-class data descriptor (Deep-MCDD) designed to identify spherical decision boundaries for each type of attack. Moreover, blockchain technology was incorporated to enhance FL, improving the security and decentralization of the framework's training process. Beuran et al. [21] proposed a novel FL-based IoT intrusion detection method by combining a shrink autoencoder and a centroid one-class classifier to build a semi-supervised learning model that effectively detects network anomalies. To address the data privacy and computational resource issues of traditional centralized learning, a FL framework is adopted, allowing each IoT gateway to train models locally without sharing raw data.

After carefully reviewing the related research literature, we identified several limitations. First, the solutions have high communication overhead, which makes them unsuitable for resource-constrained IoV environments. Second, model training performance is suboptimal when dealing with Non-IID data. Finally, in FL and blockchain-integrated solutions, the issue of node load is often overlooked. To address these issues, we propose a secure IoV intrusion detection framework that combines FD and blockchain technology to collaboratively tackle network attack problems, while also protecting privacy and ensuring system reliability.

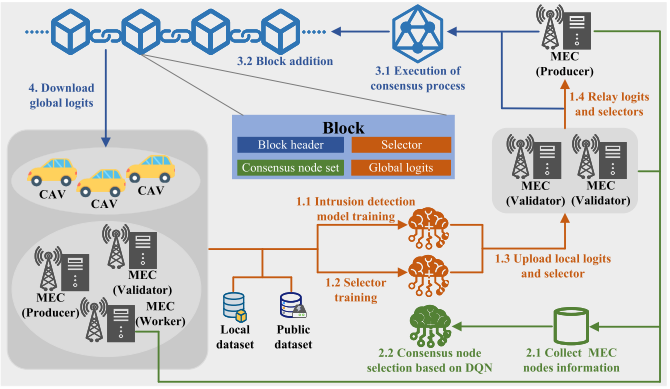

Fig. 1. Network model.

III. Network Model

The IoV intrusion detection architecture in this paper consists of three entities: Connected and Automated Vehicles (CAVs), Multi-access Edge Computing (MEC) servers, and Blockchain. The network model is shown in Fig 1.

CAVs: As FD clients, cavs train intrusion detection models using their local private datasets and make predictions on the unlabeled public dataset to generate local logits. They also train a selector based on density ratio estimation. Furthermore, cavs upload the local logits and the selector to the MEC validator. When a cav approaches the MEC, it can download and read the latest block of the blockchain to obtain the latest global logits.

MECs: MECs play a crucial role in verifying and aggregating the local logits received from FD clients. Each MEC participates in the FD process and calculates local logits and selectors. After the DQN-based node selection, an MEC is assigned one of three roles: worker, validator, or producer. As a worker, it only participates in the FD process and does not take part in the consensus process. Similar to cavs, workers send the calculated local logits and selectors to validators. As a validator, the MEC participates in both the FD and consensus processes. In the FD process, the validator organizes and forwards the local logits and selectors to the producer. In the consensus process, the validator ensures the accuracy of transactions and verifies the blocks proposed by the producer. As a producer, the MEC is selected from the validators. In the FD process, the producer computes the global logits, and in the consensus process, it is responsible for creating new blocks and performing the block chaining operation.

Blockchain: To ensure the security and transparency of the FD process, the framework uses blockchain technology to share and aggregate logits. By establishing a consortium blockchain, a trustworthy and tamper-proof ledger is created

TABLE II
CNN MODEL

Layer	Unit	Output Size
Input Layer	Input	(80, 23, 5)
Convolution Layer 1-4	Conv 1D (64, 3, 1)	(80, 64, 5)
Convolution Layer 5-6	Conv 1D (128, 3, 1)	(80, 128, 5)
Convolution Layer 7	Conv 1D (128, 3, 2)	(80, 128, 3)
Convolution Layer 8	Conv 1D (128, 3, 2)	(80, 128, 2)
Fully-Connected Layer	Linear	(80, 128)
Output Layer	Output	(80, 11)

to securely store and distribute global logits. The decentralized nature of blockchain ensures that there is no single central server controlling the entire network, thereby reducing the risks of single points of failure and data tampering.

IV. Intrusion Detection Scheme Based on Federated Distillation

A. CNN-Based Intrusion Detection Model

The CNN model is widely employed due to its exceptional capability in feature extraction, effectively facilitating the interaction of information between adjacent time windows and features. In our study, we utilize CNN to extract deep features from traffic data packets. A detailed description of the foundational CNN model is provided in Table II. Each model consists of eight convolutional layers, designed to efficiently extract feature maps from the traffic data packets, along with two fully connected layers that output the prediction results. The input channel count of the first convolutional layer is 23, corresponding to the number of rows in the input data. The first four convolutional layers each contain 64 filters, with a kernel size of 3, while the subsequent four layers are equipped with 128 filters, maintaining the same kernel size. The output of the final convolutional layer is flattened and classified by a multilayer perceptron (MLP) consisting of three fully connected layers. The final layer contains 11 neurons, representing the total number of categories. As our research focuses on establishing an efficient federated training framework, the description of the CNN-based model structure is kept concise. It is worth noting that our federated training approach does not rely on the aggregation of model parameters, allowing the system to function seamlessly even when clients employ varying model architectures.

B. Federated Distillation Scheme

A key challenge faced by federated distillation is the potential for misleading predictions due to the lack of adequately trained teacher models. Local models are prone to overfitting on their local datasets, resulting in poor performance on public dataset samples that lie outside the local distribution, especially when there is non-IID data across clients. To address this, we draw upon the work of [22] and design a selector for each client based on density ratio estimation. This selector identifies out-of-distribution samples in the public dataset, effectively filtering out misleading predictions and enhancing detection accuracy for other clients.

We assume that the system consists of K vehicles and MEC servers. For simplicity, in the following, we refer to both vehicles and MEC servers as FD clients. Each client $k \in \{1, 2, \ldots, K\}$ holds two datasets: 1) a private labeled dataset $D^{p,k} = \{(x_i^{p,k}, y_i^{p,k}) | i = 1, 2, \ldots, I^p\}$ a shared unlabeled public dataset $D^o = \{x_j^o | j = 1, 2, \ldots, I^o\}$ that is common to all clients. For a classification task with C classes (where $c \in C$), $y_i^{p,k}$ is a one-hot vector. Algorithm 1 illustrates the overall flow of the proposed scheme, and the detailed description is as follows.

Algorithm 1 Federated Distillation Scheme

Input: Private datasets $D^{p,k}$, Unlabeled public dataset D^o, client number K
Output: model $\theta^{p,k}$

```
for each client k in parallel do
    Train the local model θ^{p,k} via Eq. 1;
    for each x_j^o in D^o do
        Compute ŷ_j^{o,k} via Eq. 2;
    end for
    The selector d_k based on density ratio estimation is obtained from Eq. 3 and Eq. 4;
end for
for x_j^o ∈ D^o do
    Each client votes on the sample x_j^o based on the selector d_k;
    The weights W_j assigned by each client to the sample x_j^o are determined according to Eq. 6;
end for
The global logits are obtained according to Eq. 7;
The client performs distillation training via Eq. 8 to obtain θ^{p,k};
return: θ^{p,k};
```

Step 1: Local model training. First, each client k trains a local model $\theta^{p,k}$ using the model and private dataset described in Section IV-A. The update rule is given by:

$$\theta^{p,k} = \theta^{p,k} - \eta \nabla \psi \left(\widehat{Y}^{p,k}, Y^{p,k} \right) \tag{1}$$

In this step, $\Psi(\cdot, \cdot)$ represents the loss function to be minimized, $\widehat{Y}^{p,k}$ represents the output of the model function F, i.e., $\widehat{Y}^{p,k} = F\left(X^{p,k} \mid \theta^{p,k}\right)$, and $Y^{p,k}$ denotes the true labels of the samples. The learning rate is denoted by η. For multi-class classification tasks, the loss function is typically the cross-entropy loss.

Step 2: Prediction stage. Based on the locally trained model from the previous step, each client predicts the local logits, i.e., the labels for data samples in the shared unlabeled public dataset. Specifically, given the model $\theta^{p,k}$ and a sample $x_j^o \in D^o$, the prediction is made as follows:

$$\hat{y}_j^{o,k} = F\left(x_j^o \mid \theta^{p,k}\right) \quad x_j^o \in D^o \tag{2}$$

Step 3: Client selector construction. Assume that the input space X is compact, and define U as a uniform distribution over X with a probability density function $u(x)$. The probability density function for client k is denoted as $p_k(x)$. Our objective is to estimate the density ratio based on the observed samples:

$$d_k^* = \frac{p_k(x)}{u(x)} \tag{3}$$

Specifically, for samples x from the local distribution of client k where $p_k(x) > 0$, the density ratio $d_k^* > 0$, whereas for

out-of-distribution samples x where $p_k(x) = 0$, the density ratio $d_k^* = 0$. Therefore, the client can identify out-of-distribution samples in the public data set by constructing a density ratio estimator.

To ensure statistical convergence, the kernelized Unconstrained Least Squares Importance Fitting (KuLSIF) algorithm is used to estimate the density ratio. The KuLSIF estimation model is defined in a Reproducing Kernel Hilbert Space (RKHS) $\mathcal{D}_{\mathrm{k}}$ equipped with a Gaussian kernel function. The objective function for d_k is given by:

$$d_k = \operatorname{argmin}_{d_k' \in \mathcal{D}_{\mathrm{k}}} \frac{1}{2n_u} \sum_{x \sim D_u} \left(d_k'(x)\right)^2 - \frac{1}{n_k} \sum_{x \sim D^{p,k}} d_k'(x) + \frac{\beta}{2} \left\| d_k' \right\|_{\mathcal{D}_{\mathrm{k}}}^2 \tag{4}$$

Here, $D^{p,k}$ is the local dataset of client k with size n_k, and D_u is the sample set from the uniform distribution with size n_u. β is the regularization parameter that controls the model complexity, and $\mathcal{D}_{\mathrm{k}}$ is the RKHS, which defines the sample representation and function space within the space.

The KuLSIF algorithm minimizes the objective function to find the density ratio d_k. The optimal solution d_k^* corresponds to the minimum value of this objective function. By using kernel tricks, the algorithm optimizes in the RKHS, effectively estimating the density ratio.

Step 4: Sample weight calculation. To determine whether a sample from the public dataset is an out-of-distribution sample for client k, we use the threshold τ_k. For each sample x_j^o in the public dataset D^o, if $d_k\left(x_j^o\right)$ is less than τ_k, the sample is considered to be out-of-distribution, and client k will not upload its local prediction, as these predictions may mislead the entire system. For sample x_j^o, each client has:

$$v_j^k = \begin{cases} 1, & d_k\left(x_j^o\right) > \tau_k \\ 0, & d_k\left(x_j^o\right) < \tau_k \end{cases} \tag{5}$$

The v_j^k values from each client are aggregated to form V_j, representing the collective vote of all clients regarding whether sample x_j^o belongs to their distribution. The weight of client k for sample x_j^o is then computed as follows, where K^v is the number of clients for which $v_j^k = 1$:

$$W_j = V_j / K^v \tag{6}$$

Step 5: Aggregation phase. By gathering all local predictions from the clients $\widehat{Y}^o = \left\{\widehat{Y}^{o,k} \mid k = 1, 2, \ldots, K\right\}$, along with the weights $W = \left\{W_j \mid j = 1, 2, \ldots, I^o\right\}$ for the public dataset samples, the global logits can be computed through the following calculation:

$$\widehat{y}_j^{o,g} = \widehat{Y}_j^o \times W_j \tag{7}$$

where $\widehat{Y}_j^o$ contains all client predictions for sample x_j^o, and W_j contains the corresponding weights assigned by all clients for that sample.

Step 6: Distillation training. The global logits are distributed back to the clients, and each client's local model uses the global logits from the public dataset for distillation training. The process is as follows:

$$\theta^{p,k} = \theta^{p,k} - \eta \nabla \psi \left(x_j^o, \hat{y}_j^{o,g} \mid \theta^{p,k}\right) \tag{8}$$

TABLE III
SYMBOL TABLE

Symbol	Description
B	Block size
F_i	Node i CPU cycle frequency
P_i	Node i throughput
CON	Consensus node set
$\theta^{p,k}$	Client k intrusion detection model
$\widehat{Y}^{o,k}$	Client k local logits
d_k	Client k selector
o	Vehicle request operation
t	Timestamp
c	Vehicle identification
m	Vehicle request content
i	Validator number
v	View number
n	Message number
d	Hash value of the message content
s_c	Signature of messages by vehicle
s_p	Signature of messages by producer
s_i	Signature of messages by validator i

V. SYSTEM STRUCTURE

To ensure the security and decentralization of the FD model, we employ blockchain technology. The primary symbols used in this section are outlined in Table III. In our framework, global logits are stored on a single blockchain. The integration of blockchain with FD necessitates careful selection of both the blockchain type and consensus protocol, as these significantly impact scalability, latency, complexity, and cost. Public blockchains face scalability limitations and high latency issues, while private blockchains improve scalability and reduce latency but may compromise decentralization and privacy. Consortium blockchains offer a balanced solution, making them highly suitable for our system. In this consortium blockchain, we enhance the classical PBFT protocol by using the DQN algorithm to select consensus nodes, ensuring load balancing and significantly improving consensus efficiency. Our approach strategically prioritizes scalability, latency, and security, with a slight compromise on decentralization. We consider two types of nodes in the blockchain network:

Light Nodes: Serving as FD clients, cavs download and read blocks but do not participate in the consensus process.

Full Nodes: Refers to MECs that actively participate in both the FD and consensus processes. All MECs are involved in the FD process, but only two types of roles participate in the consensus process. In each round, each MEC is assigned one of the following roles: Worker, Validator, or Producer.

- Worker: Participates only in the FD process and does not engage in the consensus process.
- Validator: Participates in both FD and the consensus process. In the FD process, the validator first trains the local model and predicts using the public dataset. The validator also receives the local logits and selectors from light nodes and workers, organizes them, and forwards them to the producer. In the consensus process, the validator's

primary task is to verify the correctness of transactions and validate the blocks proposed by the producer.

- Producer: Participates in both FD and the consensus process and is selected from the validators. In the FD process, the producer trains the model and makes predictions on the dataset. Based on the local logits and selectors received from the validators, the producer calculates the global logits. In the consensus process, the producer is responsible for packaging the global logits and other data into a block, performing the consensus process, and committing the block to the blockchain.

A. Smart Contracts

We will consider the traffic management department, which aims to manage a privacy-preserving federated distillation process. First, the department creates and deploys a smart contract on the blockchain. Then, through the smart contract, FD clients (such as vehicles) are added to the FD system. The contract includes the client addresses and other relevant information. The smart contract provides the department (i.e., the contract owner) with the flexibility to easily add or remove FD clients, while managing the FD system in a fully decentralized, trusted, and transparent manner. The smart contract in this paper is written in Solidity. For brevity, only two important functions of the smart contract are discussed below: adding and removing FD clients, where *Col* represents an instance of an FD client; *now* denotes the current time (in seconds); *sender* is the address of the FD client sending the transaction to the smart contract; and Externally Owned Account (EOA) refers to an account controlled by a public-private key pair. *col.isOnline* indicates whether the client is online, and if it is online with a request to join the FD system, the client will be added; if it is offline for a certain period, it will be removed.

AddFDCollaborator(Col.EOA, col.info): This function can only be called by the traffic management department to add an FD client. It takes the FD client's EOA (Col.EOA) and client information (col.info) as inputs, and adds the client to the federated distillation system. RemoveFDCollaborator(Col.EOA): This function can also only be called by the traffic management department. If the client has been offline for a certain period, this function removes the FD client from the federated distillation system. It takes the FD client's EOA as input and removes the client from the system. After executing the smart contract, col.info will be output as the result. Algorithm 2 presents the core details and main structure of the contract.

Algorithm 2 FedDistSmartContract

Input: Col.EOA, col.info, col.isOnline, col.request_action
Output: col.info

```
if msg.sender is not owner || isFDCollaborator(col.EOA) == true then
    throw;
end if
if col.isOnline && col.request_action == "add_FD" then
    length ← FDColsAdr.push(col.EOA)
    FDCols[col.EOA] ← FDCol(col.EOA, col.Info, now, length-1)
    emit AddFDCollaborator(Col.EOA, col.info)
    numberOfFDCols++
else
    rowToDelete ← FDCols[col.EOA].index
    keyToMove ← FDColsAdr[length-1]
    FDColsAdr[rowToDelete] = keyToMove
    FDCols[keyToMove].index = rowToDelete
    FDColsAdr.length–
    emit RemoveFDCollaborator(Col.EOA)
    numberOfFDCols–
end if
return: col.info;
```

B. Consensus Node Selection Algorithm Based on DQN

To better integrate blockchain with FD, it is essential to enhance the classical PBFT consensus algorithm. The primary improvement involves limiting the number of nodes participating in the consensus process, thereby increasing consensus efficiency. To make the algorithm more suitable for the requirements of FD systems, we draw upon reference [23] and introduce a DQN-based consensus node selection algorithm. When blockchain networks are combined with federated distillation, the selection of consensus nodes becomes particularly crucial, as FD tasks demand significant computational resources from the nodes. Therefore, the selection of consensus nodes must consider various factors, such as the communication capabilities of nodes, their computational resources, and the impact of block size on the time overhead during the validation process. With this in mind, we employ the DQN algorithm to dynamically select consensus nodes within the blockchain network. The goal is to achieve load balancing within the blockchain-based federated distillation framework. This strategy not only helps alleviate the load on the nodes participating in the FD process, but also minimizes the time cost of the consensus procedure. The detailed process of consensus node selection is as follows.

Step 1: Node modeling. Assume there are M full nodes in the system. We model the time overhead of the consensus process and use the DQN algorithm to select the consensus set $CON = \{con_1, con_2, \ldots, con_i\}$. The total time overhead consists of two parts: the time taken by the node to validate the block, denoted as T_{valid}, and the communication time between the current node and the next node, denoted as T_{com}. Thus, the total time overhead can be represented as:

$$T = T_{\text{valid}} + T_{\text{com}} \tag{9}$$

The node validation time T_{valid} and communication time T_{com} are expressed as follows:

$$T_{\text{valid}} = \frac{B \cdot G_i}{F_i} \tag{10}$$

$$T_{\text{com}} = \frac{B}{P_i} \tag{11}$$

where B is the block size (in bytes), G_i is the number of CPU cycles required by node i to validate each byte, F_i is

the CPU cycle frequency of node i during consensus (in Hz), P_i is the remaining throughput of node i during consensus (in bits per second). Since in real-world environments, factors like latency, packet loss, and protocol overhead prevent nodes from reaching their theoretical maximum transmission capacity (i.e., bandwidth), the throughput is chosen to represent the actual transmission capability of the node.

The variables in the above equations account for the residual resources of the nodes after the federated distillation process. Based on this, T can be further refined as:

$$T = \frac{B \cdot G_i}{F_i} + \frac{B}{P_i} \quad \left(p_i < p_{i\text{-tol}}, i \in M\right) \tag{12}$$

where $p_{i\text{-tol}}$ represents the total throughput of the node.

Step 2: Optimization objective. Since federated distillation tasks and blockchain consensus processes occur simultaneously, and bandwidth is a limited resource, the optimization objective is to minimize the time required for a node to execute the consensus process. To achieve this, reinforcement learning can be applied to select the most appropriate consensus nodes in order to reduce the overall system time overhead. To optimize the selection of consensus nodes, the problem is modeled as a Markov Decision Process (MDP), consisting of the following components:

State space S: The state space of the system is represented by the resource status of each node.

$$S = \left\{F^t, P^t, B^t\right\} \tag{13}$$

where F^t represents the independent CPU cycle frequency of each node at time t, P^t represents the throughput available for consensus tasks at time t, B^t represents the data volume in the block at time t.

Action space A: At time t, the action space consists of a set of consensus node selection strategies.

$$A = \{n_1, n_2, n_3, \ldots, n_i\} \tag{14}$$

where the value of n_i is either 0 or 1. If a node is selected as a consensus node, the corresponding value is 1; otherwise, the value is 0.

Reward function $R(S, A)$: The reward function is used to measure the effectiveness of selecting a particular combination of consensus nodes. We define the reward as the inverse of the time spent by the nodes in the consensus process:

$$R(S, A) = \frac{1}{T_{\text{valid}} + T_{\text{com}}} \tag{15}$$

Here, the reward is the reciprocal of the time required for the node to execute the consensus process. The shorter the time, the higher the reward.

Leaming policy π^*: The objective of policy optimization is typically to maximize the expected cumulative reward, where γ^t is the discount factor.

$$\pi^* = \text{argmax} E\left[\sum_{t=0}^{\infty} \gamma^t R^t\right] \tag{16}$$

Step 3: Optimization algorithm. To solve this problem, we use a Deep Q-Network (DQN) to learn the optimal node selection strategy. DQN is a reinforcement learning algorithm based on Q-leaming, which can estimate the Q-values through deep neural networks and iteratively optimize the policy.

Q function: DQN estimates the expected return for taking a specific action given a state using the Q -function. The formula is:

$$Q = \left(S^t, A^t, \theta\right) \tag{17}$$

where θ represents the parameters of the neural network, and $Q\left(S^t, A^t, \theta\right)$ is the Q-value of taking action A^t in state S^t.

Target Q-value Q_{target} : DQN uses the target Q-value to compute the loss function. The target Q-value represents the estimated retum based on the reward and the maximum Q-value of the next state. The formula is:

$$Q_{\text{target}} = R\left(S^t, A^t\right) + \gamma^t \cdot \max Q\left(S^{t+1}, A^{t+1}, \hat{\theta}\right) \tag{18}$$

Loss function: The DQN loss function minimizes the prediction error of the Q-values. The loss function is defined as:

$$L(\theta) = E\left[\left(Q\left(S^t, A^t, \theta\right) - Q_{\text{target}}\right)^2\right] \tag{19}$$

where θ and $\hat{\theta}$ represent the parameters of the evaluation network and the target network, respectively. γ^t is the discount factor, with a value between 0 and 1. The objective of this loss function is to optimize the network parameters θ such that the Q -value predictions are as close as possible to the target Q-value.

Through the training process of the DQN algorithm, the neural network learns an optimal strategy π^* that selects the best combination of consensus nodes for each state. Ultimately, the DQN network will determine the consensus set $CON = \{con_1, con_2, \ldots, con_i\}$, where the nodes in this set are selected as consensus nodes. With this, the consensus node selection process is complete, and the next section will provide a detailed description of the consensus execution process.

C. Intrusion Detection Framework Based on Federated Distillation and DQN-PBFT

The proposed framework consists of three main steps: federated distillation, consensus node selection, and the execution of the consensus process. Through these steps, vehicles will obtain an intrusion detection model. It is important to note that both vehicles and MEC entities are FD clients, and both participate in the FD process. However, the term node refers solely to MEC entities, as vehicles do not engage in the consensus process. Fig 2 illustrates the overall system framework, while Fig 3 provides the system's operational timeline. The following is a detailed description of each step.

Step 1.1: Determining FD clients. The traffic management authority creates and deploys a smart contract on the blockchain, which facilitates the addition and removal of clients. Initially, when a vehicle or MEC entity submits an externally owned account (EOA) along with relevant details (such as hardware configuration, bandwidth, latency, etc.) to the traffic management authority, the smart contract is invoked to incorporate them into the FD system. Subsequently, the smart contract periodically checks the online status of the system's clients and removes those that have been offline for an extended period. Finally, the participating FD clients are confirmed.

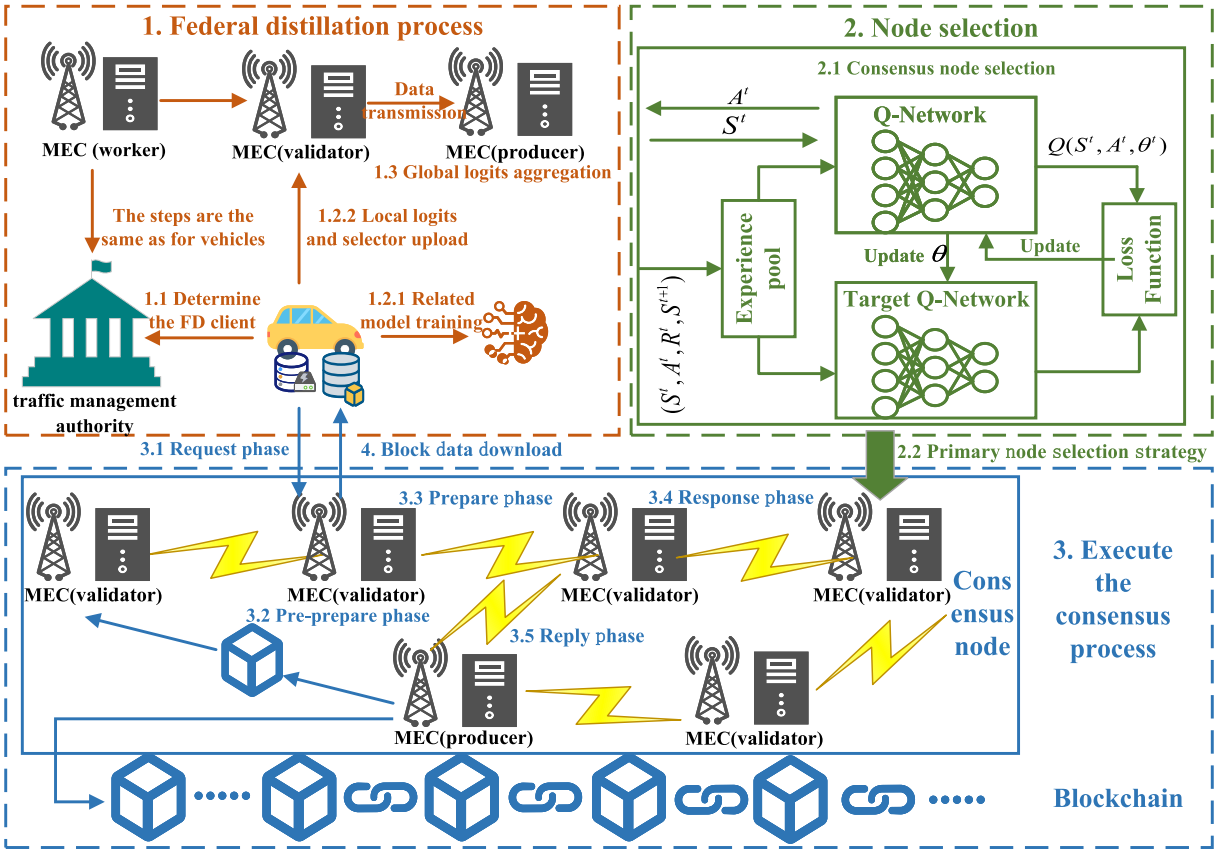

Fig. 2. System overall framework.

Step 1.2: Local training of clients. $Vehicle_k$ trains an intrusion detection model $\theta^{p,k}$ using its local private dataset and performs predictions on the public dataset to obtain local logits $\widehat{Y}^{o,k}$. Simultaneously, $Vehicle_k$ trains a selector d_k based on density ratio estimation. The vehicle then packages $\widehat{Y}^{o,k}$ and d_k and sends them to the nearest MEC validator for the current round.

Step 1.3: Global logits aggregation. After completing the model training, the validators package the collected $\widehat{Y}^{o,k}$ and d_k, and send them to the MEC producer for the current round. The producer receives the local predictions from all FD clients, $\widehat{Y}^{o} = \left\{\widehat{Y}^{o,k} \mid k = 1, 2, \ldots, K\right\}$, along with the selectors $D = \{d_k \mid k = 1, 2, \ldots, K\}$. Using the public dataset and selector D, the producer calculates the weights W (where $w_{k,j} \in W$ represents the weight of client k for sample j). The producer then computes the global logits $\widehat{Y}^{o,g}$ based on $\widehat{Y}^{o}$ and W. The detailed process of federated distillation is presented in Section IV-B.

Step 2.1: Consensus node selection. This step aims to determine the consensus nodes for the next round, rather than the current one. Since MEC nodes participate in both the FD process and the consensus process, it is necessary to ensure load balancing while optimizing consensus efficiency. Based on the previous step, we model the remaining resources of the nodes (including communication and computational resources) with the goal of minimizing consensus time. By applying the DQN algorithm, we determine the set of consensus nodes $CON = \{con_1, con_2, \ldots, con_i\}$. The specific steps are detailed in Section V-B. In this process, the nodes participating in consensus are treated as MEC validators, while those not selected are considered MEC workers.

Step 2.2: Primary node selection strategy. In traditional PBFT consensus algorithms, the primary node is elected through a round-robin approach, with the primary node's number determined by the formula $p = v \bmod n$. However, in the IoV environment, the presence of malicious nodes and attackers makes the round-robin method predictable, thereby reducing security. To address this issue, this scheme employs a Verifiable Random Function (VRF) [24] to select the primary node from the consensus nodes, with the chosen primary node serving as the MEC producer.

Step 3.1: Request phase. After completing the FD process, $Vehicle_k$ sends a message $< REQUEST, o, t, c, s_c >$ requesting the global logits to the nearest validator for the current round. The validator then forwards the message to the producer.

Step 3.2: Pre-prepare phase. Upon receiving the request from $Vehicle_k$, the producer packages the previously calculated global logits $\widehat{Y}^{o,g}$, the consensus set CON, and other relevant data into a block B, and enters the pre-prepare phase. The producer broadcasts the message $\ll PRE-PREPARE, v, n, d, B > s_p, m >$ to all validators. When a validator receives the pre-prepare message, it will validate the message. If the validation is successful, it proceeds to the prepare phase. If the validation fails, no further action is taken.

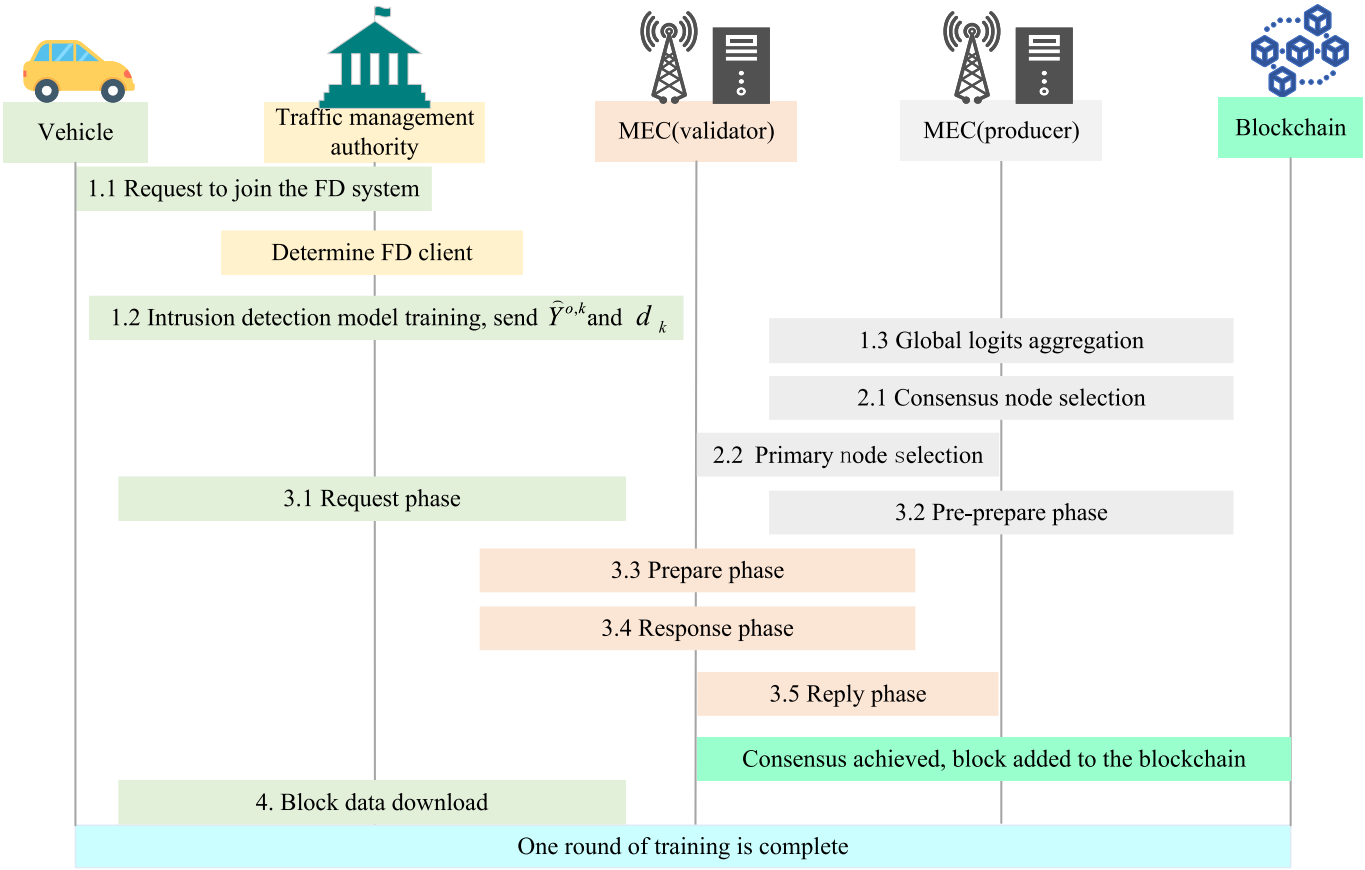

Fig. 3. System operation sequence diagram.

Step 3.3: Prepare phase. Upon entering the prepare phase, the validator broadcasts the prepare message $\ll PREPARE, v, n, d, B, i > s_i >$. While sending the prepare message, the validator will also receive and validate prepare messages broadcasted by other validators.

Step 3.4: Response phase. If a validator receives more than $2f + 1$ prepare messages, it enters the response phase, then verifies the messages and provides the results to the producer.

Step 3.5: Reply phase. When the producer receives more than $2f + 1$ identical confirmation messages, consensus is considered to be reached. The producer then provides the confirmation result to all MEC objects (including workers). At this point, the consensus process is complete. After receiving the confirmation result, all MEC objects add the block to the blockchain.

Step 4: Block data download. The validator returns the request result to $Vehicle_k$. If the consensus is successful, $Vehicle_k$ downloads the block containing the global logits from the validator, which is then used for distillation training. If the consensus fails, the system moves on to the next round of training.

VI. Experiment and Analysis

The proposed scheme primarily relies on a federated distillation-based intrusion detection model and an improved DQN-PBFT consensus algorithm. To validate the performance advantages of this approach, we first conduct experiments on the federated distillation-based intrusion detection model, followed by a detailed performance analysis and comparison. Subsequently, for the DQN-PBFT consensus algorithm, a simulation environment is set up and compared with the latest research solutions. Through these two sets of experiments, the feasibility of the proposed scheme is demonstrated.

A. Performance Analysis of Intrusion Detection Model

1) Dataset Description and Data Preprocessing: The N-BaIoT [25] dataset is a publicly available benchmark dataset, comprising nine sub-datasets from nine Internet of Things (IoT) devices. Among these nine devices, seven support 11 types of traffic, including one normal traffic type and ten attack traffic types. The remaining two devices handle six types of traffic, including one normal traffic type and five attack traffic types. Each traffic packet contains 115 features, which are extracted from five distinct time windows (100 milliseconds, 500 milliseconds, 1.5 seconds, 10 seconds, and 1 minute). These features can be quickly computed, meeting the real-time detection requirements for malicious traffic packets. Due to the dataset's coverage of a wide range of traffic types and its rich traffic records, the N-BaIoT dataset has been widely used in the field of intrusion detection and has become a benchmark dataset. The data preprocessing is performed in the same way as described in [8], consisting of three main steps: data partitioning, normalization, and two-dimensional transformation.

1)Data Partitioning: In practical applications, there are numerous clients (e.g., vehicles) participating in FD, and the data is typically non-IID. To accommodate this, we partition the original dataset. Since directly using the original dataset requires substantial resources, we constructed the mini-N-BaIoT dataset, which includes 11 types of traffic data from nine IoT devices. Specifically, the nine sub-datasets of the original dataset are represented as $D_1, D_2, \ldots, D_9$. Each D_i is divided into C_i subsets based on traffic type, where each subset $D_{i,c}$ contains only the traffic data of type c. We selected 1,000 traffic records from each subset $D_{i,c}$ to form the $D_{i,l}^{\text{mini}}$ in the mini-N-BaIoT dataset. These new datasets are then split into private datasets D^p, public datasets D^o, and test datasets D^{test} with a ratio of 70%, 10%, and 20%, respectively. It is important to note that these datasets are mutually independent, and D^o does not contain labels. All data records in D^p belong to D_i, and D^p is assigned to K clients for experimentation.

Non-IID partitioning: D^p is sorted according to its classification labels, with a total of 11 classes. These 11 classes are randomly divided into three groups in the form of 4,4, and 3, and distributed among three clients. The data distribution is visualized in Fig 4 (red represents the public dataset, and green represents the client's private datasets). After the dataset partitioning, the distribution of each dataset satisfies the following: different clients have significantly different categories, exhibiting the non-IID characteristics that align with real-world IoV environments. There is no overlap between the public dataset D^o, private dataset D^p, and test dataset D^{test} .

2) Normalization: The feature dimensions of the traffic data vary significantly, leading to large differences in values. To enhance the training effectiveness of the model, we apply min-max normalization to scale the feature values between 0 and 1.

3) Dimensionalization: Each sample x_i consists of 115 features, which are divided into five parts based on time windows. We transform the sample features into a matrix of five columns and 23 rows, making it suitable for input into the subsequent CNN model, as follows:

$$x_i = \begin{bmatrix} x_{i,0} & x_{i,23} & x_{i,46} & x_{i,69} & x_{i,92} \\ x_{i,1} & x_{i,24} & x_{i,47} & x_{i,70} & x_{i,93} \\ x_{i,2} & x_{i,25} & x_{i,48} & x_{i,71} & x_{i,94} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{i,22} & x_{i,45} & x_{i,68} & x_{i,91} & x_{i,114} \end{bmatrix} \tag{20}$$

2) Experimental Setup: To ensure a clear evaluation of the experiment, the following sections describe the experimental environment, implementation details, and evaluation metrics:

1) Experimental environment: All evaluations are conducted in Python 3.7, using the PyTorch framework version 1.9.0, running on a machine equipped with an Intel Core i5-13600KF @ 3.50 GHz, 32GB RAM, and an NVIDIA GeForce RTX A4000 GPU.

2) Implementation details: During the training phase, the Adam optimizer is employed. The learning rate, batch size, and the number of local training epochs per communication round are set to 0.0001, 100, and 5, respectively. When constructing the client selector, clients reserve a portion of their local data as a validation set. The threshold τ_k for the client selector is set as the first quartile of the estimated ratio on the validation set.

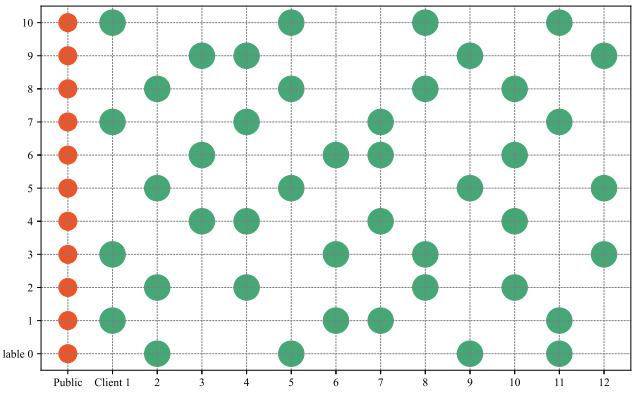

Fig. 4. Non-IID data visualization.

3) Evaluation metrics: To assess the performance of the method, we calculate the numbers of true positives (T_p), true negatives (T_n), false positives (F_p), and false negatives (F_n). Based on these definitions, we derive the recall, precision, and F1-score.

$$\text{Recall} = \frac{T_p}{T_p + F_n} \tag{21}$$

$$\text{Precision} = \frac{T_p}{T_p + F_p} \tag{22}$$

$$F1 - \text{score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{23}$$

3) Model Performance Evaluation: In this experiment, we comprehensively compared the detection performance of different approaches by evaluating their accuracy, precision, and F1-score, with the specific results presented in Fig 5. We compared the proposed method with two representative federated distillation methods—DS-FL [7] and FedMd [26]. Similar to our approach, both of these methods rely on the public dataset for knowledge transfer. According to the experimental results, the highest accuracy achieved by the DS-FL method was 51.33%, while FedMd reached 55.95%. In contrast, the proposed method achieved a maximum accuracy of 81.46%, with a precision of 82.72% and an F1-score of 79.42%. It is evident that our method demonstrates significant performance improvements across all metrics, particularly in terms of accuracy and precision, far surpassing the other two approaches.

The significant improvement in performance can be attributed to the fact that both the DS-FL and FedMd methods employed a simple averaging-based weighted approach for aggregating global logits. However, when facing the non-IID issue of client data, this simplistic weighting strategy led to clients uploading misleading knowledge, which negatively impacted the global model's training performance. In particular, the DS-FL method further compounded this issue by applying entropy-based reduction after simple weighting, which intensified the inconsistency between data, rendering the training process more inefficient and unstable. In contrast, the proposed method designs a client selector based on density

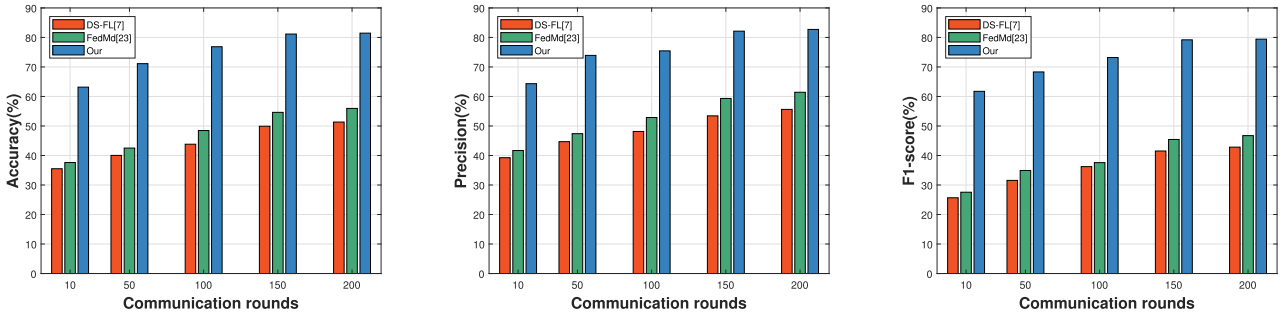

Fig. 5. Comparison of model performance. (a) Accuracy; (b) Precision; (c) F1-score.

ratio estimation, which effectively identifies out-of-distribution samples in the public dataset. For these out-of-distribution samples, we set their aggregation weight to zero, effectively preventing the upload of misleading information from clients and shielding the training process of other clients from such negative influences. This strategy ensures that only reliable knowledge from trustworthy data distributions is shared during the federated distillation process, thus enhancing the accuracy and robustness of the global logits, particularly in the face of various types of data biases.

To further analyze the performance of the proposed method, we plotted the confusion matrix on the test dataset (as shown in Fig 6). The results clearly demonstrate that our federated distillation approach performs exceptionally well in handling network attacks, effectively identifying different types of network attack traffic. Notably, the G_TCP and G_UDP attack types tend to be confused with each other, mainly due to their classification as DoS attacks, and their high similarity in certain features. This observation suggests that although these attacks differ in behavioral patterns, shared network features lead to difficulties in classification. Therefore, when designing intrusion detection systems, it is crucial to further optimize feature extraction and modeling strategies to address these challenges.

B. DQN-PBFT Simulation and Analysis

We implemented the related algorithms based on PyTorch 1.9.0 and used NS-3 to simulate the network communication environment. This setup simulated network parameters such as latency, bandwidth, CPU rate, and reputation values, which were fed back in real-time to the respective models. In the simulation environment, we ran four consensus protocols: PBFT [27], SG-PBFT [28], R-PBFT [29], and DQN-PBFT, and evaluated their consensus latency, throughput, and communication times under varying numbers of MEC nodes.

Consensus latency refers to the time required for the system to reach consensus after a node proposes a block. It is a key metric for evaluating the responsiveness and efficiency of a consensus algorithm. Throughput, on the other hand, represents the number of transactions or blocks the system can process in a given time period, often measured in transactions per second (TPS), reflecting the efficiency of the consensus algorithm in handling transactions. Communication times

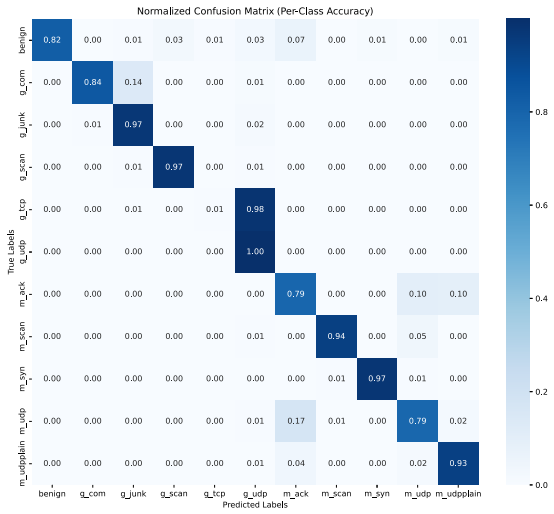

Fig. 6. Confusion matrix.

refers to the number of message exchanges between nodes during the consensus process, used to assess the communication complexity of the consensus protocol.

During the experimental process, we considered various parameters such as network delay, bandwidth, CPU speed, and reputation values of MEC nodes. The SG-PBFT algorithm selects $N/2$ (where N is the total number of nodes) consensus nodes based on the nodes' scores, while the R-PBFT algorithm chooses $N/3$ consensus nodes based on reputation values. In contrast, our approach uses parameters such as bandwidth and CPU speed as inputs and employs the DQN algorithm to dynamically select i nodes as consensus nodes.

As shown in Fig 7, as the number of MEC nodes increases, the consensus delay of each consensus algorithm gradually rises, throughput begins to decline, and the number of communication times increases. This is because all four consensus algorithms rely on communication between nodes to reach consensus. As the number of MEC nodes increases, the communication overhead also grows, leading to longer times for nodes to reach consensus, which results in a reduction in the number of transactions processed per unit time. Compared to the classical PBFT algorithm, both SG-PBFT and R-PBFT improve the performance of the three metrics by limiting the number of consensus nodes based on scores and reputation values.

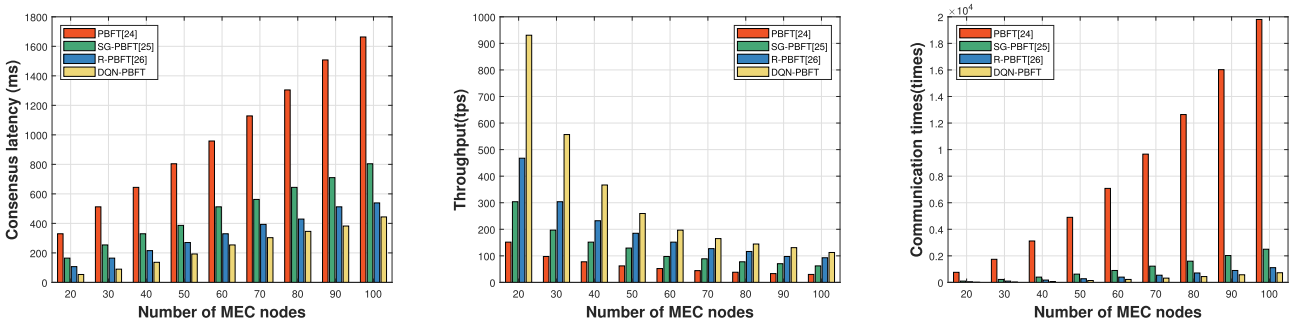

Fig. 7. Comparison of consensus performance. (a) Consensus latency; (b) Throughput; (c) Communication times.

The DQN-PBFT algorithm proposed in this paper also enhances consensus efficiency by limiting the number of consensus nodes, but it is particularly designed for environments combining federated distillation with blockchain. In such environments, MEC nodes are not only required to participate in consensus but also need to undertake the federated distillation task. Therefore, the selection of consensus nodes is crucial and must ensure load balancing across nodes. By using parameters like bandwidth and CPU speed as inputs, DQN-PBFT dynamically selects i MEC nodes as consensus nodes using the DQN algorithm, effectively ensuring load balancing. As a result, DQN-PBFT outperforms SG-PBFT and R-PBFT in terms of consensus efficiency. In contrast, the SG-PBFT and R-PBFT algorithms, which select consensus nodes based on scores and reputation values, are not suitable for the federated distillation environment. If the consensus nodes selected by these algorithms cannot handle both tasks simultaneously, the system's consensus efficiency will significantly decrease.

VII. Conclusion

In this paper, we propose an intrusion detection solution for the IoV that combines federated distillation and blockchain technology, leveraging a public dataset to facilitate knowledge transfer between clients. First, to address the non-IID issue of IoV data, we design a density ratio-based selector for each FD client to prevent the sharing of misleading knowledge with other clients. Next, we introduce blockchain technology into the federated distillation system, using smart contracts to determine the clients participating in federated distillation. To further optimize system performance, we propose the DQN-PBFT consensus algorithm, which intelligently selects appropriate clients as consensus nodes. By reducing the number of nodes involved in the consensus process, this mechanism significantly improves consensus efficiency. Additionally, it enhances load balancing and fairness among clients in both the federated distillation and blockchain processes. Finally, through comparative experiments on federated distillation and consensus algorithms, we validate the effectiveness and feasibility of the proposed method.

Although the proposed approach demonstrates strong performance in detecting known attacks, it still exhibits certain limitations when dealing with unknown attacks. To enhance the system's generalization capability and overall security defense, future research will focus more on the identification of unknown threats. In addition, to improve the system's adaptability against emerging threats, we plan to incorporate a dynamically updatable FD mechanism, enabling each client to continuously learn and effectively recognize novel attack patterns.

References

[1] N. Hu, Y. Jia, M. Zhang, Y. Li, H. Zhao, and L. Luo, "Security assessment of intelligent connected vehicles based on the cyber range," *IEEE Netw.*, vol. 38, no. 3, pp. 57–62, May 2024.

[2] B. Lampe and W. Meng, "Intrusion detection in the automotive domain: A comprehensive review," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 4, pp. 2356–2426, 2023.

[3] S. Rajapaksha, H. Kalutarage, M. O. Al-Kadri, A. Petrovski, G. Madzudzo, and M. Cheah, "AI-based intrusion detection systems for in-vehicle networks: A survey," *ACM Comput. Surveys*, vol. 55, no. 11, pp. 1–40, Nov. 2023.

[4] R. Ahmad, I. Alsmadi, W. Alhamdani, and L. Tawalbeh, "Zero-day attack detection: A systematic literature review," *Artif. Intell. Rev.*, vol. 56, no. 10, pp. 10733–10811, Oct. 2023.

[5] Z. A. E. Houda, A. S. Hafid, and L. Khoukhi, "MiTFed: A privacy preserving collaborative network attack mitigation framework based on federated learning using SDN and blockchain," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 4, pp. 1985–2001, Jul. 2023.

[6] N. Rodríguez-Barroso, D. Jiménez-López, M. V. Luzón, F. Herrera, and E. Martínez-Cámara, "Survey on federated learning threats: Concepts, taxonomy on attacks and defences, experimental study and challenges," *Inf. Fusion*, vol. 90, pp. 148–173, Feb. 2023.

[7] S. Itahara, T. Nishio, Y. Koda, M. Morikura, and K. Yamamoto, "Distillation-based semi-supervised federated learning for communication-efficient collaborative training with non-IID private data," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 191–205, Jan. 2023.

[8] R. Zhao, Y. Wang, Z. Xue, T. Ohtsuki, B. Adebisi, and G. Gui, "Semisupervised federated-learning-based intrusion detection method for Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8645–8657, May 2023.

[9] T. Qi, F. Wu, C. Wu, L. He, Y. Huang, and X. Xie, "Differentially private knowledge transfer for federated learning," *Nature Commun.*, vol. 14, no. 1, p. 3785, Jun. 2023.

[10] Z. Yu, Y. Liu, G. Xie, R. Li, S. Liu, and L. T. Yang, "TCE-IDS: Time interval conditional entropy- based intrusion detection system for automotive controller area networks," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1185–1195, Feb. 2023.

[11] Z. Deng, J. Liu, Y. Xun, and J. Qin, "IdentifierIDS: A practical voltage-based intrusion detection system for real in-vehicle networks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 661–676, 2024.

[12] L. Du, Z. Gu, Y. Wang, and C. Gao, "Open world intrusion detection: An open set recognition method for CAN bus in intelligent connected vehicles," *IEEE Netw.*, vol. 38, no. 3, pp. 76–82, May 2024.

[13] M. S. Korium, M. Saber, A. Beattie, A. Narayanan, S. Sahoo, and P. H. J. Nardelli, "Intrusion detection system for cyberattacks in the Internet of Vehicles environment," *Ad Hoc Netw.*, vol. 153, Feb. 2024, Art. no. 103330.

[14] P. Waghmode, M. Kanumuri, H. El-Ocla, and T. Boyle, "Intrusion detection system based on machine learning using least square support vector machine," *Sci. Rep.*, vol. 15, no. 1, p. 12066, Apr. 2025.

[15] W. Lo, H. Alqahtani, K. Thakur, A. Almadhor, S. Chander, and G. Kumar, "A hybrid deep learning based intrusion detection system using spatial–temporal representation of in-vehicle network traffic," *Veh. Commun.*, vol. 35, Jun. 2022, Art. no. 100471.
[16] K. Agrawal, T. Alladi, A. Agrawal, V. Chamola, and A. Benslimane, "NovelADS: A novel anomaly detection system for intra-vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 22596–22606, Nov. 2022.
[17] X. Li, Z. Hu, M. Xu, Y. Wang, and J. Ma, "Transfer learning based intrusion detection scheme for Internet of Vehicles," *Inf. Sci.*, vol. 547, pp. 119–135, Feb. 2021.
[18] Z. A. E. Houda, H. Moudoud, B. Brik, and L. Khoukhi, "Blockchain-enabled federated learning for enhanced collaborative intrusion detection in vehicular edge computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 7661–7672, Jan. 2024.
[19] (2022). *PySyft*. [Online]. Available: https://github.com/OpenMined/PySyft
[20] A. A. Korba, A. Boualouache, and Y. Ghamri-Doudane, "Zero-X: A blockchain-enabled open-set federated learning framework for zero-day attack detection in IoV," *IEEE Trans. Veh. Technol.*, vol. 73, no. 9, pp. 12399–12414, Apr. 2024.
[21] V. T. Nguyen and R. Beuran, "FedMSE: Semi-supervised federated learning approach for IoT network intrusion detection," *Comput. Secur.*, vol. 151, Apr. 2025, Art. no. 104337.
[22] J. Shao, F. Wu, and J. Zhang, "Selective knowledge sharing for privacy-preserving federated distillation without a good teacher," *Nature Commun.*, vol. 15, no. 1, Jan. 2024, Art. no. 349, doi: 10.1038/s41467-023-44383-9.
[23] X. Zhou et al., "Federated distillation and blockchain empowered secure knowledge sharing for Internet of Medical Things," *Inf. Sci.*, vol. 662, Mar. 2024, Art. no. 120217.
[24] X. Zhang, R. Li, and H. Zhao, "A parallel consensus mechanism using PBFT based on DAG-lattice structure in the Internet of Vehicles," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 5418–5433, Mar. 2023.
[25] Y. Meidan et al., "N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, Jan. 2018.
[26] D. Li and J. Wang, "FedMD: Heterogenous federated learning via model distillation," 2019, *arXiv:1910.03581*.
[27] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. OsDI*, 1999, pp. 173–186.
[28] G. Xu et al., "SG-PBFT: A secure and highly efficient distributed blockchain PBFT consensus algorithm for intelligent Internet of Vehicles," *J. Parallel Distrib. Comput.*, vol. 164, pp. 1–11, Jun. 2022.
[29] A. Kumar, L. Vishwakarma, and D. Das, "R-PBFT: A secure and intelligent consensus algorithm for Internet of Vehicles," *Veh. Commun.*, vol. 41, Jun. 2023, Art. no. 100609.

Wenxian Jiang (Senior Member, IEEE) is currently an Associate Professor. His research interests include blockchain security and the IoVs. He is an ACM Member.

Zhenping Guo was born in 2000. He is currently pursuing the M.S. degree in electronic information with Huaqiao University, Xiamen, China. His research interests include network communications, blockchain, and the IoVs.

Naizhou Wang was born in 2000. He received the B.S. degree in computer science and technology from Fujian Normal University, Fuzhou, China. He is currently pursuing the M.S. degree in computer technology with Huaqiao University, Xiamen, China. His research interests include network security, blockchain, and the Internet of Things.